

STM32 ARM Assembly Lab: LED Blink and Blue Button Input

Target Hardware: NUCLEO-L476RG Development Board

Objective: Write pure ARM assembly code to blink the onboard user LED (LD2 on PA5) and read the blue user button (B1 on PC13) as a digital input.

Duration: 3-4 hours

Lab Overview

In this lab, you will:

- Set up an STM32CubeIDE project configured for assembly programming
- Learn the minimum hardware configuration steps to control GPIO
- Write ARM Cortex-M4 assembly code to toggle an LED with a visible delay

Configure GPIOC pin PC13 as an input for the onboard blue user button (B1)

Read GPIOC_IDR, apply active-low button logic, and branch based on the button state

- Build, flash, and verify your program on the NUCLEO-L476RG board

Why assembly? Understanding low-level register manipulation gives you deep insight into how microcontrollers work and prepares you for advanced embedded systems programming.

Project Requirements

Software

- **STM32CubeIDE** (for writing, building, and debugging your code)
- Download from: <https://www.st.com/en/development-tools/stm32cubeide.html>
- Installation video: https://www.youtube.com/watch?v=qv0303qv03y_L-fgA

Hardware

- **NUCLEO-L476RG** development board
- **USB Type-A to Mini-B cable** (to connect the board to your computer)

Required Documents (PDFs)

- **Cortex-M4 Devices Generic User Guide** (ARM) - for ARM instruction set reference
- **STM32L476 Reference Manual (RM0351)** - for MCU register details and peripheral configuration
- **NUCLEO-L476RG User Manual (UM1724)** - CRITICAL: contains board schematic showing LED/button wiring and pin connections
- **STM32L476 Datasheet** (optional) - additional electrical specifications

Important: The Nucleo board schematic in UM1724 shows you which MCU pins are connected to the LED, button, and Arduino headers. The MCU reference manual (RM0351) tells you how to configure those pins via registers.

All documents are available from STMicroelectronics documentation portal at:
<https://www.st.com/en/evaluation-tools/nucleo-l476rg.html>

Installing STM32CubeIDE

STM32CubeIDE is a free, Eclipse-based integrated development environment for STM32 microcontrollers. Follow the installation steps for your operating system.

Download STM32CubeIDE

1. Go to: <https://www.st.com/en/development-tools/stm32cubeide.html>
2. Click **Get Software**
3. Select your operating system (Windows, macOS, or Linux)
4. You may need to create a free ST account or log in
5. Download the installer (approximately 700 MB - 1 GB)

Windows Installation

1. Double-click the downloaded .exe installer file
2. Click **Yes** when prompted by User Account Control
3. Click **Next** on the welcome screen
4. Accept the license agreement and click **Next**
5. Choose installation directory (default: C:\ST\STM32CubeIDE) or browse to change
6. Click **Next**, then **Install**
7. Wait for installation to complete (3-5 minutes)
8. Click **Finish** to launch STM32CubeIDE

The installer automatically includes ST-LINK drivers for board communication.

Installation video tutorial for windows:

https://www.youtube.com/watch?v=qv0303qv03y_L-fgA

macOS Installation

1. Open the downloaded .dmg disk image file
2. **First:** Install ST-LINK Server
 - Double-click the `st-stlink-server` icon
 - If blocked, go to **System Preferences** → **Security & Privacy**
 - Click the padlock to unlock, then click **Open Anyway**
 - Complete the ST-LINK Server installation
3. **Second:** Install STM32CubeIDE
 - Drag the **STM32CubeIDE** icon to the **Applications** folder
 - If you get "damaged" error when launching, open Terminal and run:

```
sudo xattr -r -d com.apple.quarantine /Applications/STM32CubeIDE.app
```

3. Launch STM32CubeIDE from Launchpad or Applications folder

Linux Installation

1. Extract the downloaded archive:

```
tar -xvzf st-stm32cubeide_*.sh.zip
```

2. Run the installer script with sudo:

```
sudo sh ./st-stm32cubeide_*_amd64.sh
```

3. Follow the console prompts to complete installation
4. Add your user to the dialout group for USB access:

```
sudo usermod -a -G dialout $USER
```

5. Log out and log back in for group changes to take effect
6. Launch STM32CubeIDE from applications menu or run:

```
/opt/st/stm32cubeide/stm32cubeide
```

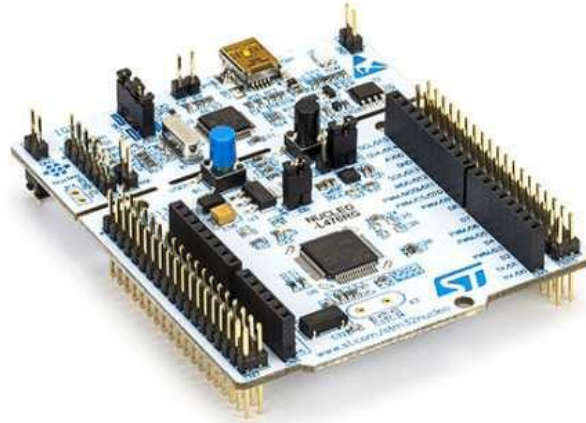
First Launch Setup

When you first launch STM32CubeIDE:

1. **Workspace Selection:** Choose a directory for your projects (e.g., `Documents/STM32Workspace`)
 2. Click **Launch**
 3. Close the welcome screen
 4. You're ready to create projects!
-

Hardware Background

NUCLEO-L476RG Board Overview



The NUCLEO-L476RG is a low-cost development board featuring:

- **MCU:** STM32L476RGT6 (ARM Cortex-M4 @ 80 MHz)
- **Flash:** 1 MB
- **RAM:** 128 KB
- **Programmer/Debugger:** ST-LINK/V2-1 (on-board, no external tool needed)
- **Arduino Uno Rev3 connectivity:** Compatible headers
- **Morpho headers:** All STM32 I/O pins accessible

LED Connection

The NUCLEO-L476RG board has a green user LED labeled **LD2** connected to **GPIO Port A, Pin 5 (PA5)**.

- When PA5 is driven **HIGH** (logic 1), the LED turns **ON**
- When PA5 is driven **LOW** (logic 0), the LED turns **OFF**
- The LED is active-high (no current-limiting resistor needed - built into board)

How to find this information:

Open the NUCLEO-L476RG User Manual (UM1724) page 24

Or:

Go to the schematic section ,find the LED labeled LD2 and,Trace the net to find the connected MCU pin: PA5.

User Button: The board also has a blue user button labeled B1 connected to PC13. This improved lab uses B1 in Part 5. On the standard Nucleo-64 configuration, B1 is active-low: PC13 reads 1 when the button is released and 0 when the button is pressed. Students will verify this behavior in the debugger.

Understanding Memory-Mapped I/O

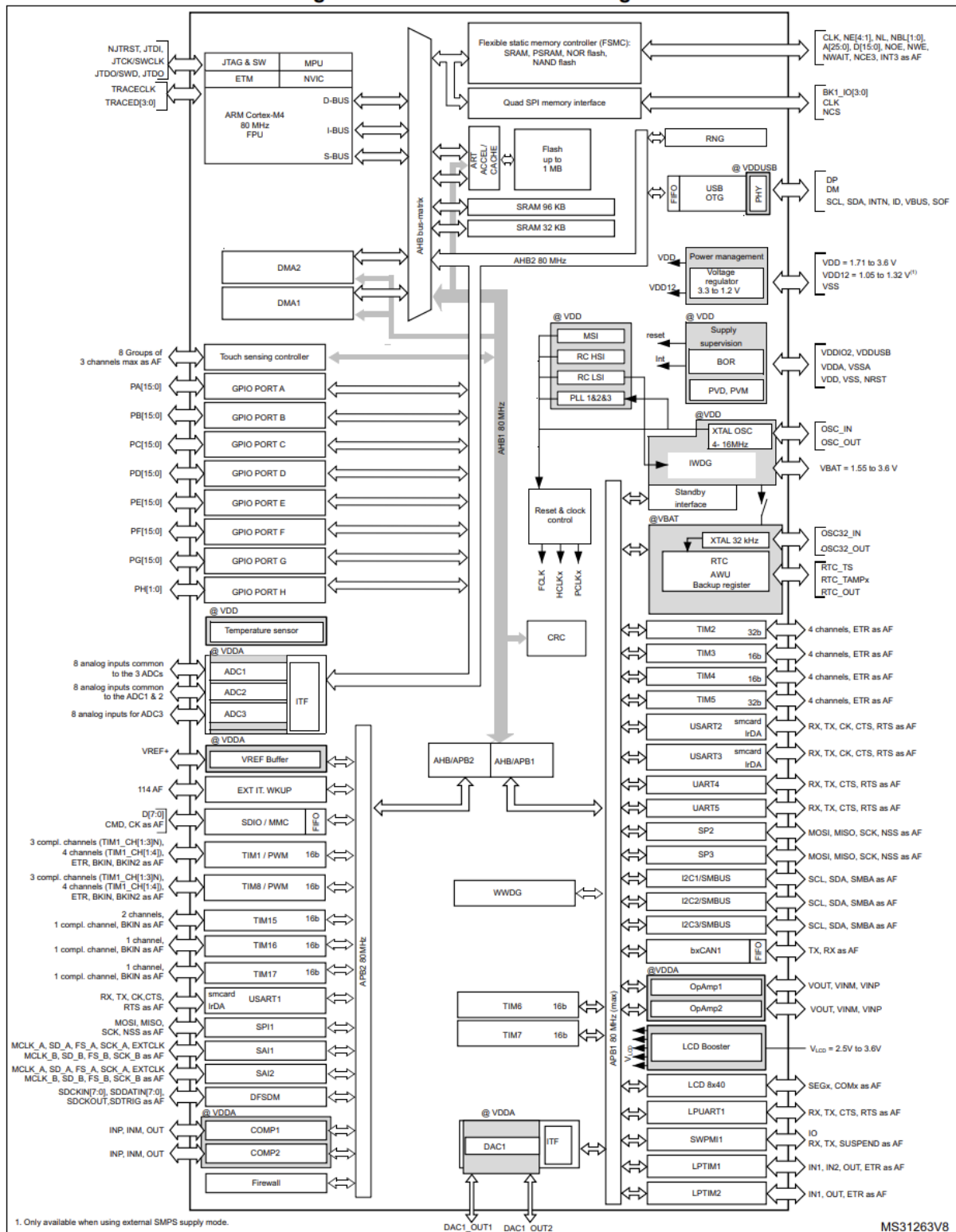
STM32 microcontrollers use **memory-mapped I/O**, meaning hardware peripherals are controlled by reading and writing specific memory addresses.

Region	Address Range	Contents
Flash Memory	0x08000000 - 0x080FFFFFFF	Your program code (1 MB)
SRAM	0x20000000 - 0x2001FFFF	Variables and stack (128 KB)
Peripherals	0x40000000 - 0x5FFFFFFF	Hardware registers (GPIO, UART, etc.)

Table 1: STM32L476RG main memory regions

To identify how to configure the peripheral , we need to see which bus is attached to our peripheral .Open the Datasheet page 17 .

Figure 1. STM32L476xx block diagram



GPIOA is attached to AHB2 bus, so it's address must be located in the AHB2 domain.

Go to the reference manual page 83 , to confirm that GPIOA is in the AHB2 domain.

By looking at the addresses boundaries, each GPIO port occupies 1 KB of address space. For **GPIOA**, the base address is **0x48000000**.

	0x4800 0400 - 0x4800 07FF	1 KB	GPIOB	Section 8.5.13: GPIO register map
	0x4800 0000 - 0x4800 03FF	1 KB	GPIOA	Section 8.5.13: GPIO register map
-	0x4002 BC00 - 0x47FF FFFF	~127 MB	Reserved	-

In section 8.5 (page 304), we can see the register's definition for GPIOx (x=A .. H) ,each register is 32bit wide, and has an offset address from the base.

How the address is calculated : gpio register address =gpiox base + register offset

Exercice : find the address of GPIOH _PUPDR register

Answer:

How it's derived:

- PERIPH_BASE = 0x4000_0000 and AHB2PERIPH_BASE = PERIPH_BASE + 0x0800_0000 = 0x4800_0000
- GPIOH_BASE = AHB2PERIPH_BASE + 0x1C00 = 0x4800_1C00
- PUPDR offset inside each GPIO port is 0x0C

So: GPIOH_PUPDR = 0x4800_1C00 + 0x0C = 0x4800_1C0C.

GPIO Port A Register Map

Register	Offset	Address	Purpose
GPIOA_MODER	0x00	0x48000000	Pin mode (input/output/alternate/analog)
GPIOA_OTYPER	0x04	0x48000004	Output type (push-pull/open-drain)
GPIOA_OSPEEDR	0x08	0x48000008	Output speed (low/medium/high/very high)
GPIOA_PUPDR	0x0C	0x4800000C	Pull-up/pull-down configuration
GPIOA_IDR	0x10	0x48000010	Input data register (read pin states)
GPIOA_ODR	0x14	0x48000014	Output data register (write pin states)
GPIOA_BSRR	0x18	0x48000018	Bit set/reset register (atomic operations)

Table 2: GPIOA register layout (partial)

Key Registers for LED Control

To control PA5, we need three registers:

Register	Address	Purpose
RCC_AHB2ENR	0x4002104C	Enable clock to GPIOA peripheral (bit 0 = GPIOAEN)
GPIOA_MODER	0x48000000	Set PA5 as output (bits 11:10 = 01)
GPIOA_ODR	0x48000014	Control PA5 state (bit 5: 0=OFF, 1=ON)

Table 3: Essential registers for LED blink

Register Bit Fields Explained

RCC_AHB2ENR (RCC base: 0x40021000, offset: 0x4C)

Bit 7	Bit 6	...	Bit 2	Bit 1	Bit 0	...
GPIOH	GPIOG	...	GPIOC	GPIOB	GPIOA	...

Table 4: RCC_AHB2ENR bit layout

Each bit enables one GPIO port Setting bit 0 to 1 enables the clock to GPIOA. **Without enabling the clock, all writes to GPIOA registers are ignored.**

GPIOA_MODER (Address: 0x48000000)

Each pin uses 2 bits to define its mode:

- **00** = Input mode
- **01** = General-purpose output
- **10** = Alternate function
- **11** = Analog mode

For PA5 (pin 5), the mode bits are **[11:10]**:

...	Bits 11:10	Bits 9:8	Bits 7:6	...	Bits 1:0
...	PA5 mode	PA4 mode	PA3 mode	...	PA0 mode

Table 5: GPIOA_MODER bit field layout

To set PA5 as output: write **01** to bits [11:10].

GPIOA_ODR (Address: 0x48000014)

Each bit controls one pin's output state:

Bit 15	...	Bit 5	Bit 4	...	Bit 1	Bit 0
PA15	...	PA5	PA4	...	PA1	PA0

Table 6: GPIOA_ODR bit layout

- Write **1** to bit 5 → PA5 outputs HIGH (LED ON)
- Write **0** to bit 5 → PA5 outputs LOW (LED OFF)

Important: Before using any GPIO port, you must enable its clock in the RCC (Reset and Clock Control) peripheral.

Quick Reference: From Schematic to Code

1. **Consult board schematic (UM1724):** Find which MCU pin connects to your hardware
 - Example: LD2 LED → PA5
 - Example: B1 button → PC13
 2. Consult MCU reference manual (RM0351): Find register addresses and bit fields
 - Memory map chapter → find peripheral base address
 - GPIO chapter → find register offsets and bit definitions
 3. **Write assembly code:** Translate register operations into ARM instructions
-

Step-by-Step Instructions

Part 1: Create the STM32CubeIDE Project

1. Launch STM32CubeIDE

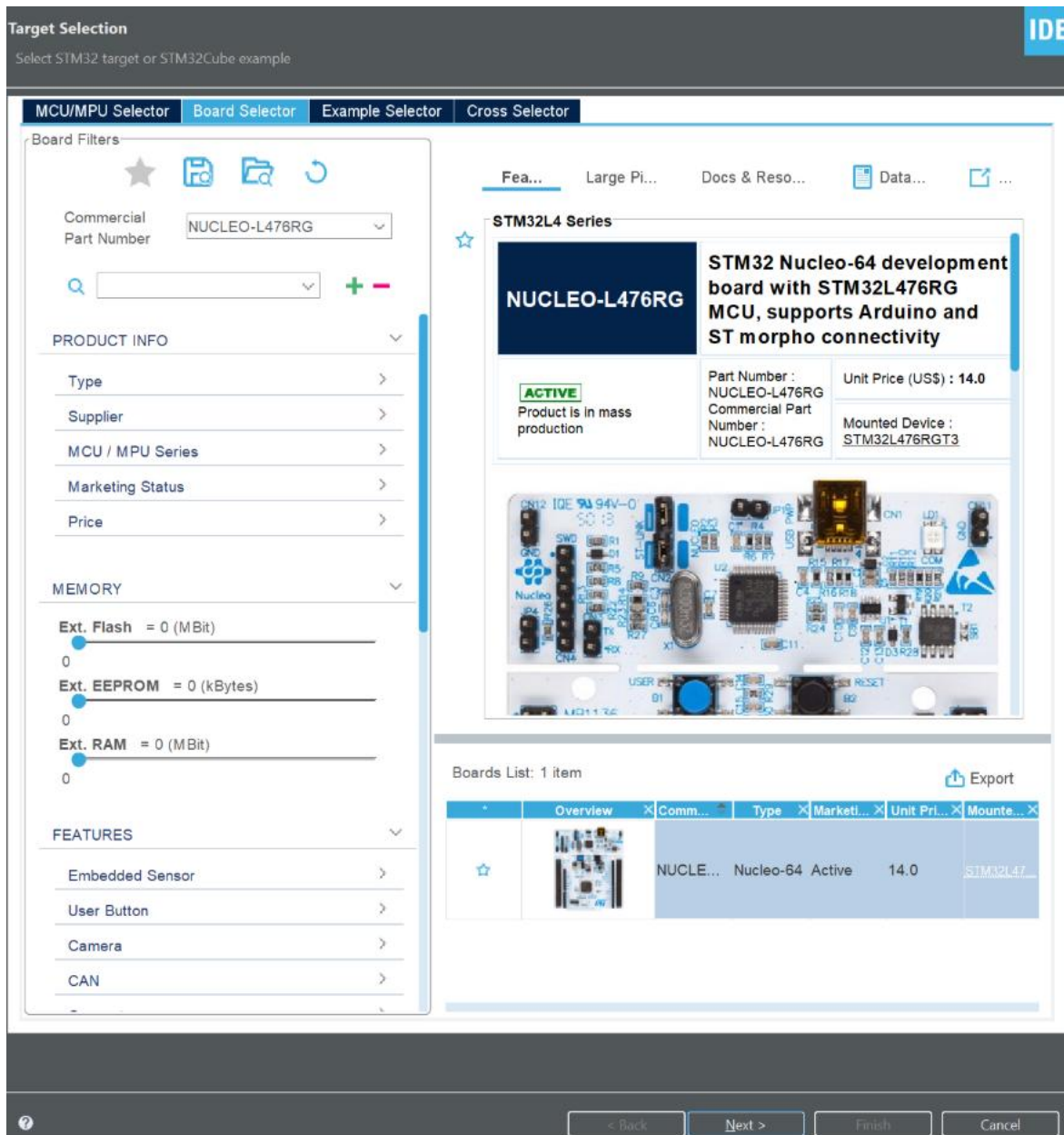
- Open STM32CubeIDE
- Select or create a workspace directory

2. Create New STM32 Project

- Go to: **File → New → STM32 Project**
- The Target Selection window opens

3. Select Board

- Click the **Board Selector** tab
- In the search field, type: NUCLEO-L476RG
- Select the **NUCLEO-L476RG** board from the results
- Click **Next**



4. Configure Project Settings

In the Setup STM32 Project dialog:

- **Project Name:** Enter a descriptive name (e.g., STM32_ASM_LED_Blink)
- Check **Use default location** or specify a custom directory
- **Targeted Language:** Select **C**
- **Targeted Binary Type:** Select **Executable**
- **Targeted Project Type:** Select **Empty** (NOT STM32Cube)

Important: Select **Empty** project type - this gives you a minimal project without HAL libraries.

Setup STM32 project

Project

Project Name: STM32_ASM_tutorial

☒ Use default location

Location: C:/TouchGFXProjects Browse...

Options

Targeted Language

☒ C ☐ C++

Targeted Device Usage

Targeted Binary Type

☒ Executable ☐ Static Library

Targeted Project Type

☐ STM32Cube ☒ Empty

? < Back Next > Finish Cancel

Why Empty project? The "STM32Cube" project type includes HAL libraries for high-level programming. We want bare-metal register access, so we use "Empty" which provides only startup code and linker scripts.

5. Finish Project Creation

- Click **Finish**
- Wait a few seconds while the IDE downloads board support files (first time only)

Part 2: Set Up the Assembly Environment

1. Remove Unnecessary C Files

In the Project Explorer on the left:

- Expand the `src` folder
- Delete: `main.c`, `syscalls.c`, and `sysmem.c`
- Keep: Startup assembly file (e.g., `startup_stm32l476rgtx.s`)

Why? We're writing pure assembly, so we don't need the C standard library support files. The startup file handles chip initialization and calls your `main` function.

In the src folder , create a new file , by right clicking on it, and then new source file, call it syscalls_stub.s and paste this code into it:

```
.syntax unified
.thumb
.cpu cortex-m4

.global _write
.thumb_func
_write:
bx lr

.global _read
.thumb_func
_read:
bx lr

.global _close
.thumb_func
_close:
bx lr

.global _lseek
.thumb_func
_lseek:
bx lr
```

In the **startup file**, comment out the instructions **b1 __libc_init_array** and **b1 SystemInit** by adding `//` at the beginning of each line, as illustrated in the figure below:

```
58 ldr r0, _estack
59 mov sp, r0          /* set stack pointer */
60 /* Call the clock system initialization function.*/
61 //b1 SystemInit
62
63 /* Copy the data segment initializers from flash to SRAM */
64 ldr r0, _sdata
```

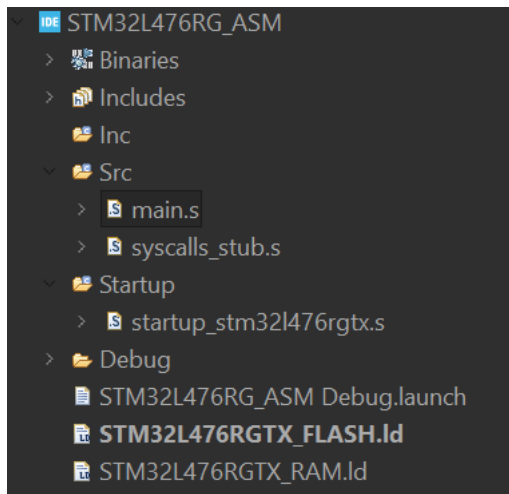
2. Understand Project Files

Your project should now contain:

- **Startup file** (startup_stm321476xx.s) - Handles reset, vector table, and calls main
- **Linker script** (.ld file) - Defines memory layout (Flash, RAM)
- **syscalls_stub.s** - Syscalls.c is where you define/implement your system calls functions like Printf() Scanf() and other C standard functions.

After that, create the file main.s with the same manner as we created syscalls_sub.s

Your project folder should be looking like this :



Part 3: Create Your Assembly Code

1. Assembly Code Template

open the arm cortex-M4 generic user guide, it has all the assembly instructions in it, and core registers explanation.

Copy the following template into `main.s`. Read the comments carefully - they explain each step.

```
.syntax unified
.thumb
.cpu cortex-m4

@ =====
@ Base addresses from STM32L476 Reference Manual
@ =====
.equ RCC_BASE,      0x40021000
.equ RCC_AHB2ENR,   0x4C
.equ GPIOA_BASE,    0x48000000
.equ GPIOA_MODER,   0x00
.equ GPIOA_ODR,     0x14

@ =====
@ Bit masks and values
@ =====
.equ RCC_AHB2ENR_GPIOAEN, (1 << 0)  @ Bit 0: enable GPIOA clock
.equ GPIO_MODER5_MASK,    (3 << 10)  @ Bits 11:10 for PA5 mode
.equ GPIO_MODER5_OUT,     (1 << 10)  @ 01 = general-purpose output
.equ GPIO_ODR5,           (1 << 5)   @ Bit 5: PA5 output data

.equ DELAY_COUNT, 0x200000          @ Delay loop count (~0.5s at 4
```

MHz)

```
@ =====  
@ Main function  
@ =====  
.global main  
.type main, %function
```

main:

```
@ -----  
@ Step 1: Enable clock for GPIOA  
@ -----  
@ Load RCC base address into r0  
LDR r0, =RCC_BASE
```

```
@ Read current RCC_AHB2ENR register value  
LDR r1, [r0, #RCC_AHB2ENR]
```

```
@ Set bit 0 (GPIOAEN) to enable GPIOA clock  
ORR r1, r1, #RCC_AHB2ENR_GPIOAEN
```

```
@ Write back to RCC_AHB2ENR  
STR r1, [r0, #RCC_AHB2ENR]
```

```
@ -----  
@ Step 2: Configure PA5 as output  
@ -----  
@ Load GPIOA base address into r0  
LDR r0, =GPIOA_BASE
```

```
@ Read current MODER register value  
LDR r1, [r0, #GPIOA_MODER]
```

```
@ Clear mode bits for PA5 (bits 11:10)  
BIC r1, r1, #GPIO_MODER5_MASK
```

```
@ Set PA5 mode to output (01)  
ORR r1, r1, #GPIO_MODER5_OUT
```

```
@ Write back to GPIOA_MODER  
STR r1, [r0, #GPIOA_MODER]
```

```
@ -----  
@ Step 3: Main loop - toggle LED forever  
@ -----
```

loop:

```
@ Read current ODR value  
LDR r1, [r0, #GPIOA_ODR]
```

```

@ Toggle bit 5 (PA5) using XOR
EOR    r1, r1, #GPIO_ODR5

@ Write back to GPIOA_ODR (LED changes state)
STR    r1, [r0, #GPIOA_ODR]

@ -----
@ Step 4: Delay
@ -----
@ Load delay count into r2
LDR    r2, =DELAY_COUNT

delay_loop:
@ Decrement counter
SUBS   r2, r2, #1

@ Branch back if not zero
BNE    delay_loop

@ Return to main loop (blink continues forever)
B      loop

@ We never reach here, but good practice:
.size  main, .-main

```

Part 4: Build and Flash

1. Build Project

- Click the **Build** button (hammer icon) in the toolbar
- Or: **Project → Build Project**
- Check the Console pane for any errors

Expected output: "Build Finished. 0 errors, 0 warnings"

2. Connect Hardware

- Connect the NUCLEO-L476RG board to your computer via USB Mini-B cable
- The ST-LINK LED should light up (red or green)
- No additional drivers needed on most systems

3. Flash Program

- Click the **Run** button (green play icon)
- Or: **Run → Run**
- CubeIDE will automatically flash the program to the board

4. Verify Operation

Success: The green LED (LD2) should blink on and off with approximately 0.5-1 second intervals.

Understanding the Code

Clock Enable (Step 1)

```
LDR r0, =RCC_BASE
LDR r1, [r0, #RCC_AHB2ENR]
ORR r1, r1, #RCC_AHB2ENR_GPIOAEN
STR r1, [r0, #RCC_AHB2ENR]
```

What it does: Enables the clock signal to the GPIOA peripheral. Without this, GPIOA registers cannot be modified.

Why? STM32 uses clock gating to save power - peripherals are off by default.

GPIO Configuration (Step 2)

```
LDR r0, =GPIOA_BASE
LDR r1, [r0, #GPIOA_MODER]
BIC r1, r1, #GPIO_MODER5_MASK
ORR r1, r1, #GPIO_MODER5_OUT
STR r1, [r0, #GPIOA_MODER]
```

What it does: Configures PA5 as a general-purpose output pin.

Details:

- Each pin has 2 bits in MODER register (00=input, 01=output, 10=alternate function, 11=analog)
- PA5 uses bits 11:10
- We clear both bits first (BIC), then set bit 10 (ORR) to get 01 (output mode)

LED Toggle (Step 3)

```
LDR r1, [r0, #GPIOA_ODR]
EOR r1, r1, #GPIO_ODR5
STR r1, [r0, #GPIOA_ODR]
```

What it does: Toggles the LED state (ON→OFF or OFF→ON).

How? The EOR (exclusive OR) instruction flips bit 5 each time:

- If bit 5 was 0, XOR with 1 makes it 1
- If bit 5 was 1, XOR with 1 makes it 0

Delay Loop (Step 4)

```
LDR r2, =DELAY_COUNT
delay_loop:
SUBS r2, r2, #1
BNE delay_loop
```

What it does: Creates a visible delay by counting down from a large number.

Why? Without delay, the LED would toggle millions of times per second - too fast to see. This simple loop makes the blink visible.

Part 5: Add Blue Button Input (B1 on PC13)

Goal: extend the LED blink program so students also read a real digital input. The blue B1 user button is connected to GPIOC pin PC13. The final behavior for this guided section is simple: LD2 is OFF when the button is released and ON when B1 is pressed.

Important active-low rule: on this Nucleo board, B1 reads 1 when released and 0 when pressed. Many beginners assume pressed equals 1; this lab intentionally highlights that the schematic determines the logic.

New hardware and register facts

Item	STM32 connection	Student meaning
LD2 green user LED	PA5	Active-high output: write 1 to PA5 to turn LED ON.
B1 blue user button	PC13	Active-low input: released = 1, pressed = 0.
GPIOA clock	RCC_AHB2ENR bit 0	Must be enabled before configuring or writing PA5.
GPIOC clock	RCC_AHB2ENR bit 2	Must be enabled before configuring or reading PC13.

GPIOC register reference for PC13

Register	Address	Bits used	Purpose
GPIOC_MODER	0x48000800	PC13 bits [27:26]	Set to 00 for input mode.
GPIOC_PUPDR	0x4800080C	PC13 bits [27:26]	Use 00 for no pull; optional 01 pull-up can be discussed.
GPIOC_IDR	0x48000810	PC13 bit 13	Read button state. 1 = released, 0 = pressed.
GPIOA_BSRR	0x48000018	PA5 set/reset bits	Atomically set or reset LED output without disturbing other PA pins.

Pre-lab questions for students

Which RCC_AHB2ENR bit enables GPIOC?

Which two GPIOC_MODER bits configure PC13?

If B1 is active-low, should the code branch to the pressed handler when PC13 bit 13 is 1 or 0?

Why is GPIOx_IDR used for input, while GPIOx_ODR or GPIOx_BSRR is used for output?

Step-by-step coding plan

Keep the LED setup from the previous blink program: enable GPIOA and configure PA5 as output.

Also enable GPIOC by setting RCC_AHB2ENR bit 2.

Configure PC13 as input by clearing bits [27:26] in GPIOC_MODER.

Read GPIOC_IDR and test bit 13 with the TST instruction.

Because the button is active-low, branch to the pressed case when the test result is zero.

Use GPIOA_BSRR to set PA5 when pressed and reset PA5 when released.

Complete guided code: button turns LED on

After the original LED-only blink works, replace main.s with this guided version. It keeps the register-level style but introduces GPIO input, active-low logic, and BSRR set/reset operations.

```
.syntax unified
.thumb
.cpu cortex-m4

@ =====
@ STM32L476RG Assembly Lab: LD2 LED + B1 Blue Button
@ LD2: PA5, active-high
@ B1 : PC13, active-low (released = 1, pressed = 0)
@ =====

.equ RCC_BASE,      0x40021000
.equ RCC_AHB2ENR,   0x4C

.equ GPIOA_BASE,    0x48000000
.equ GPIOC_BASE,    0x48000800
.equ GPIO_MODER,     0x00
.equ GPIO_PUPDR,     0x0C
.equ GPIO_IDR,       0x10
.equ GPIO_BSRR,      0x18

.equ GPIOAEN,        (1 << 0)
.equ GPIOCEN,        (1 << 2)

.equ PA5_MODER_MASK, (3 << 10)
.equ PA5_MODER_OUT,  (1 << 10)
.equ PA5_SET,         (1 << 5)
.equ PA5_RESET,       (1 << 21)    @ 5 + 16 = reset half of BSRR

.equ PC13_MODER_MASK, (3 << 26)
.equ PC13_PUPDR_MASK, (3 << 26)
.equ PC13_PIN,        (1 << 13)

.global main
.type main, %function
```

```

main:
    @ -----
    @ 1. Enable clocks for GPIOA and GPIOC
    @ -----
    LDR    r0, =RCC_BASE
    LDR    r1, [r0, #RCC_AHB2ENR]
    ORR    r1, r1, #(GPIOAEN | GPIOCEN)
    STR    r1, [r0, #RCC_AHB2ENR]

    @ Small delay after clock enable is safe practice
    NOP
    NOP

    @ -----
    @ 2. Configure PA5 as output for LD2
    @ -----
    LDR    r0, =GPIOA_BASE          @ r0 = GPIOA base
    LDR    r1, [r0, #GPIO_MODER]
    BIC    r1, r1, #PA5_MODER_MASK
    ORR    r1, r1, #PA5_MODER_OUT
    STR    r1, [r0, #GPIO_MODER]

    @ -----
    @ 3. Configure PC13 as input for B1
    @ -----
    LDR    r3, =GPIOC_BASE          @ r3 = GPIOC base
    LDR    r1, [r3, #GPIO_MODER]
    LDR    r2, =PC13_MODER_MASK
    BIC    r1, r1, r2              @ 00 = input mode
    STR    r1, [r3, #GPIO_MODER]

    @ Optional: clear PC13 pull-up/pull-down bits.
    @ The Nucleo button circuit already defines the level.
    LDR    r1, [r3, #GPIO_PUPDR]
    LDR    r2, =PC13_PUPDR_MASK
    BIC    r1, r1, r2              @ 00 = no internal pull
    STR    r1, [r3, #GPIO_PUPDR]

    @ Keep useful constants in registers for the main loop
    LDR    r4, =PC13_PIN
    LDR    r5, =PA5_SET
    LDR    r6, =PA5_RESET

loop:
    @ Read the input data register for GPIOC
    LDR    r1, [r3, #GPIO_IDR]

    @ Test PC13. If result is zero, B1 is pressed.
    TST    r1, r4
    BEQ    button_pressed

button_released:
    STR    r6, [r0, #GPIO_BSRR]    @ LED OFF
    B      loop

```

```

button_pressed:
    STR    r5, [r0, #GPIO_BSRR]    @ LED ON
    B      loop

.size main, .-main

```

Debugger checkpoint

Run the program. LD2 should be OFF when B1 is released.

Hold the blue B1 button. LD2 should turn ON.

If the LED behavior is reversed, re-check the active-low rule and the BEQ branch.

Student explanation: why BSRR is useful

The earlier LED blink program writes GPIOA_ODR. That works for one pin, but GPIOA_BSRR is safer for embedded systems because one write can set or reset a selected pin without changing the other pins in the same port. In this lab, writing bit 5 of BSRR sets PA5; writing bit 21 resets PA5.

Optional extension: B1 changes blink speed

Once the direct button-control version works, students can return to the blinking behavior and use B1 to choose between a fast and slow delay. This reinforces input reading, conditional branching, and delay loops.

```

@ Add these constants near the other .equ lines
.equ GPIO_ODR,      0x14
.equ PA5_PIN,       (1 << 5)
.equ DELAY_FAST,    0x080000
.equ DELAY_SLOW,    0x300000

@ Use this as the main loop after PA5 and PC13 are configured.
@ Assumes: r0 = GPIOA_BASE, r3 = GPIOC_BASE, r4 = PC13_PIN
loop:
    LDR    r1, [r3, #GPIO_IDR]
    TST    r1, r4
    BEQ    fast_mode          @ active-low: pressed -> fast blink

slow_mode:
    LDR    r2, =DELAY_SLOW
    B      toggle_and_delay

fast_mode:
    LDR    r2, =DELAY_FAST

toggle_and_delay:
    LDR    r1, [r0, #GPIO_ODR]
    EOR    r1, r1, #PA5_PIN
    STR    r1, [r0, #GPIO_ODR]

delay_loop:
    SUBS    r2, r2, #1
    BNE    delay_loop
    B      loop

```

Debouncing note

Mechanical push buttons can bounce, meaning the electrical signal may briefly switch several times during one physical press. For this lab, direct reading is acceptable because the LED simply follows the current button state. For a future edge-detection lab, add a short delay after detecting a press, read PC13 again, and only accept the press if the state is still active-low.

Experiments and Challenges

Basic Experiments

1. **Change blink speed**
 - Modify `DELAY_COUNT` value
 - Larger value = slower blink
 - Smaller value = faster blink
 - Try: `0x1000000`, `0x4000000`
2. **Use debugger**
 - Set breakpoint at `loop` label
 - Click Debug button
 - Step through code and watch registers change
 - View `GPIOA_ODR` in memory browser

Advanced Challenges

1. **Morse code SOS**
 - Implement: 3 short blinks, 3 long blinks, 3 short blinks
 - Use two different delay counts
2. **Button input extensions**
 - Start with the guided Part 5 button-control program, then modify it so B1 changes blink speed or toggles between two LED modes.
 - Add a short debounce delay and explain why mechanical buttons can generate multiple transitions.
 - Use the debugger or memory browser to show `GPIOC_IDR` bit 13 changing between released and pressed states.
3. **Multiple LEDs**
 - Connect external LEDs to other GPIO pins
 - Create different patterns (alternating, sequential, etc.)

Debugging (intro):

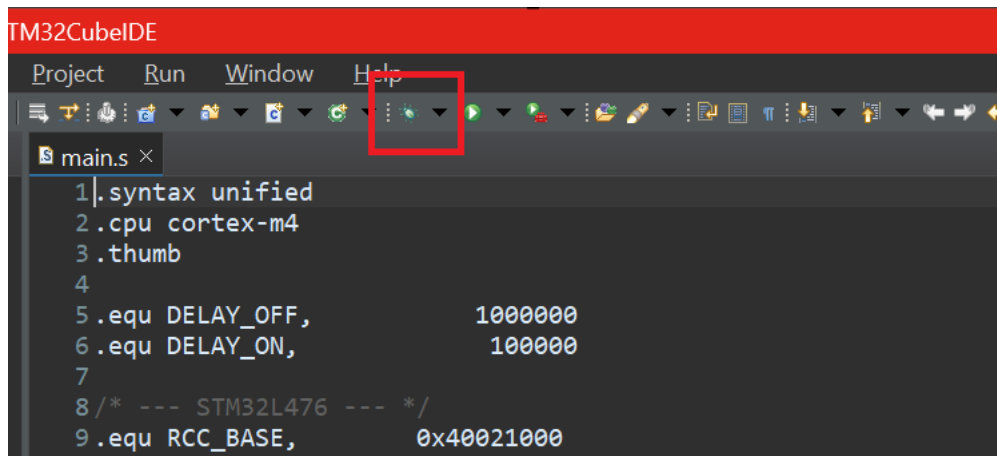
Go to this github page:

https://github.com/faridfarahmand/SSU/tree/main/MicroController/STMProjects/STM32L476RG_ASM

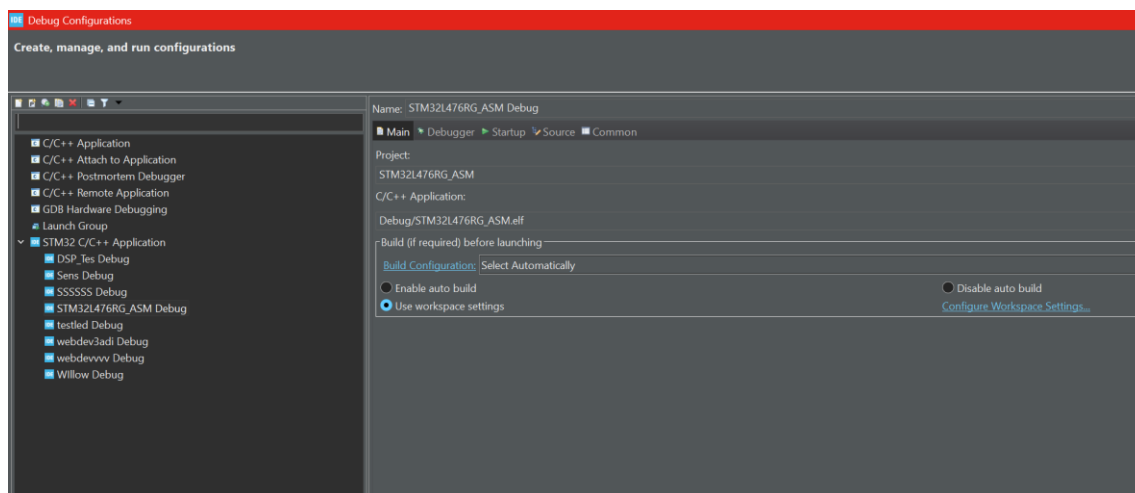
Copy the main.s content of the file and paste it in your main.s in your CubeIDE.

Let us understand the code :

Click on the arrow to expand the debug tab , and then click on debug configuration.



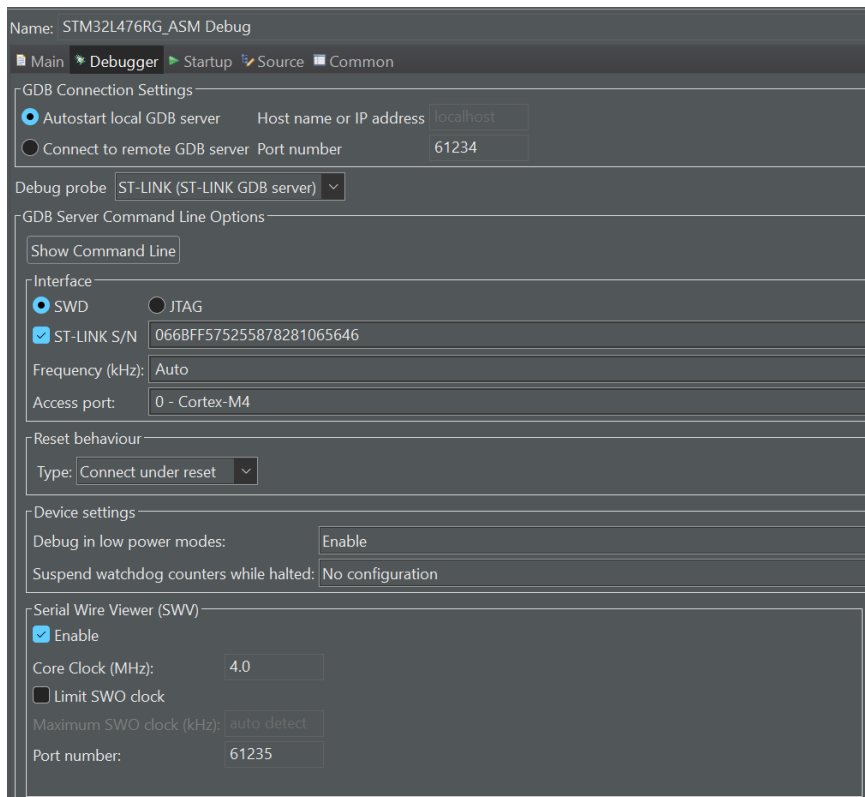
A new window appears:



Click on debugger and set as following :

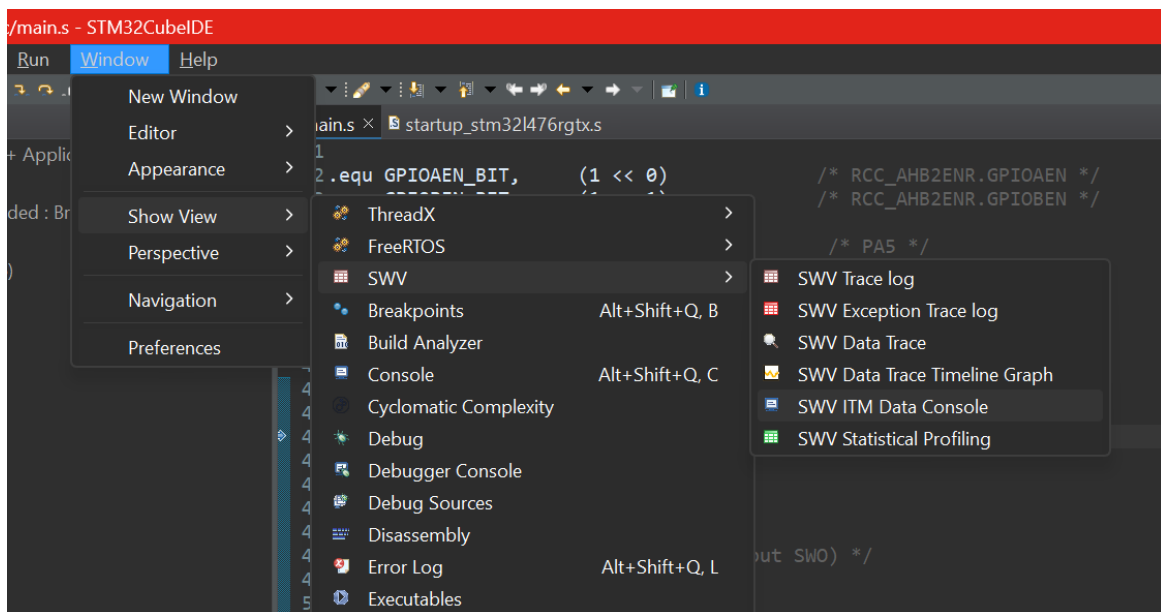
Activate the SWD interface , and enable SWV (Serial wire viewer)

Set the core clock to 4.0 Mhz

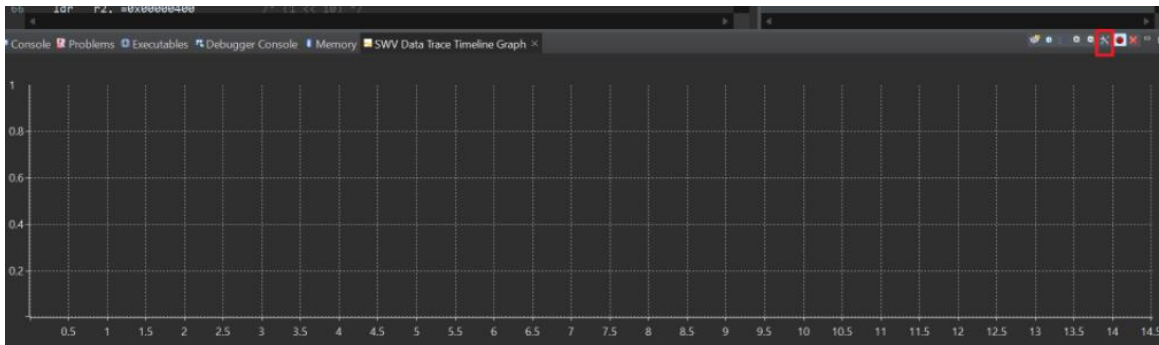


Then click on debug (the board must be attached to your computer)

After that , go to window, show view, click on SWV Data Trace Timeline Graph:



At the bottom half , you should have something looking like a scope,click on the configure trace icon:



Activate porto and enable comparator 0 , in the addr/var type in the address of GPIOA_ODR which is in our case 0x48000014.

Serial Wire Viewer settings for STM32L476RG_ASM Debug

Clock Settings
Core Clock: 4 MHz
Clock Prescaler: 2
SWO Clock: 2000.0 kHz

Trace Events
☐ CPI: Cycles per instruction ☐ EXC: Exception overhead
☐ SLEEP: Sleep cycles ☐ LSU: Load store unit cycles
☐ FOLD: Folded instructions ☐ EXETRC: Trace Exceptions

PC Sampling
☐ Enable Resolution: 16384 Cycles/sample

Timestamps
☒ Enable Prescaler: 1

Data Trace

Comparator 0
☒ Enable
Var/Addr: 0x48000014
Access: Read/Write
Size: Word
Generate: Data Value

Comparator 1
☐ Enable
Var/Addr: 0x0
Access: Read/Write
Size: Word
Generate: Data Value

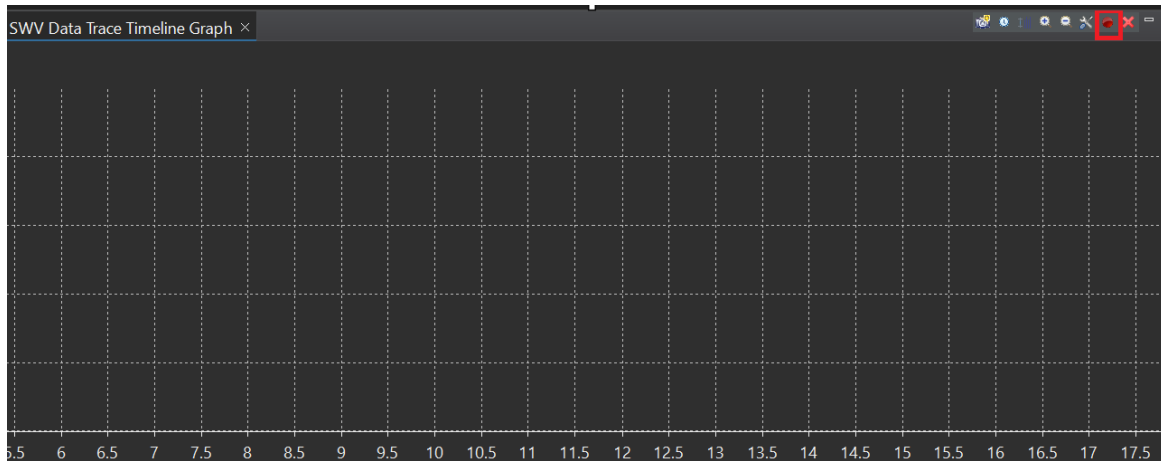
Comparator 2
☐ Enable
Var/Addr: 0x0
Access: Read/Write
Size: Word
Generate: Data Value

Comparator 3
☐ Enable
Var/Addr: 0x0
Access: Read/Write
Size: Word
Generate: Data Value

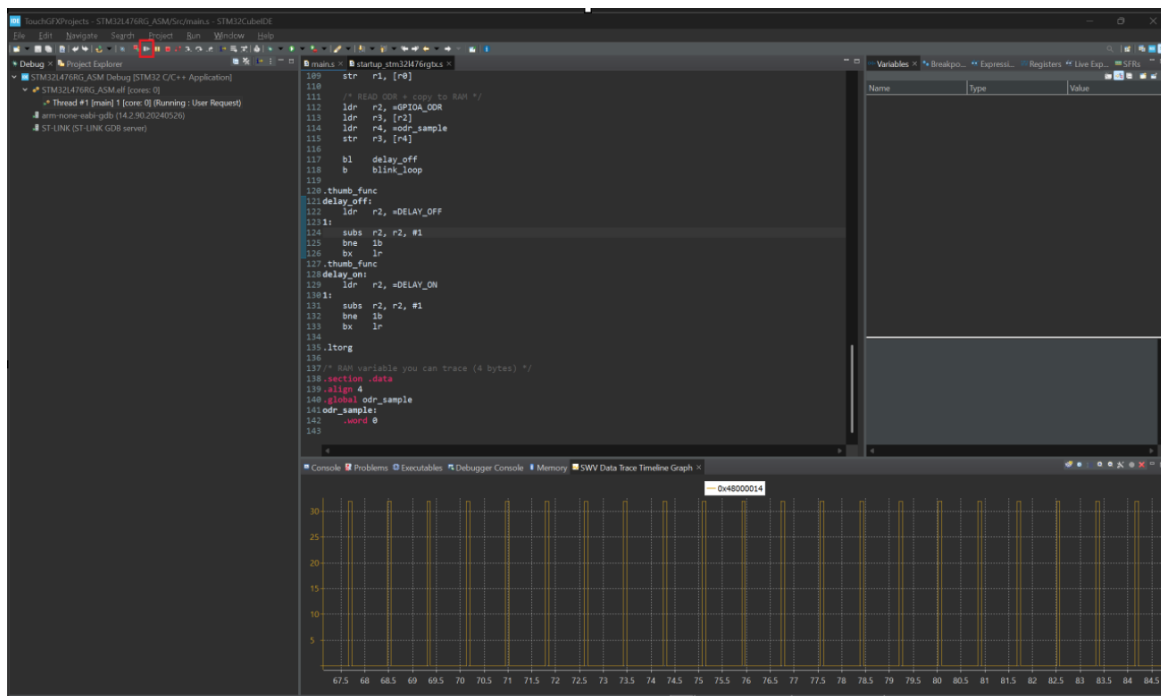
ITM Stimulus Ports
Enable port: 31 ☐ 24 ☐ 23 ☐ 16 ☐ 15 ☐ 8 ☐ 7 ☐ 0 ☒
Privileged only ports: ☐ Port 31..24 ☐ Port 23..16 ☐ Port 15..8 ☐ Port 7..0

OK Cancel

Click OK , then click on start trace



Then click on the resume icon on the top or press F8 on your keyboard, you should start seeing a rectangular wave in your console



Troubleshooting

LED Doesn't Blink

- **Check cable connection** - Mini-B USB must be firmly connected
- **Verify power** - ST-LINK LED should be lit

- **Check build output** - Must show 0 errors
- **Verify flash success** - Console should show "Download verified successfully"
- **Try reset button** - Press black reset button on board

Build Errors

- **"Undefined reference to main"** - Ensure `.global main` directive is present
- **Syntax errors** - Check that all `.equ` definitions use commas, not colons
- **Missing startup file** - Verify startup assembly file exists in project

LED Always On or Off

- **Check GPIO initialization** - Clock must be enabled before configuring GPIO
- **Verify MODER setting** - PA5 must be set to output mode (01)
- **Test with constant value** - Replace toggle with `MOV r1, GPIO_ODR5` to force ON

Lab Deliverables

Updated requirement: include a short explanation of B1/PC13 active-low behavior and identify which RCC, MODER, PUPDR, IDR, and BSRR registers your button program uses.

Submit the following:

1. Your complete `main.s` assembly source file
2. Short report (1-2 pages) including:
 - Brief explanation of each register used (RCC_AHB2ENR, GPIOA_MODER, GPIOA_ODR)
 - Description of your delay calculation
 - Photo or video of working LED blink
 - Any challenges encountered and solutions
3. (Optional) Documented code for any challenge experiments completed

Additional Resources

- STM32L476 Reference Manual (RM0351) - Complete register descriptions
 - ARM Cortex-M4 Generic User Guide - Instruction set reference
 - NUCLEO-L476RG User Manual (UM1724) - Board pinout and features
 - STM32CubeIDE User Guide - IDE features and debugging
-

Key Takeaways

By completing this lab, you should understand:

- How to configure STM32 GPIO registers for output and input in assembly language
- The importance of clock enable before peripheral access
- Register bit manipulation using ARM instructions (LDR, STR, ORR, BIC, EOR, TST, BEQ)
- Basic program structure: initialization → main loop
- How to build and flash embedded programs using STM32CubeIDE